FIDELO E-BIKE STORE

System Documentation

Inventory Management and Customer Application System

React 18 · Express · MySQL 8 · MinIO · Docker Compose

Version 1.0 · 15 August 2026

1. Introduction

Fidelo E-Bike Store is a web-based inventory management and customer application system for a multi-location e-bike business. It centralizes the product catalog, per-location stock, stock movement history, inter-branch transfers, customer product applications, notifications, operational reports, and authentication audit records.

Objectives

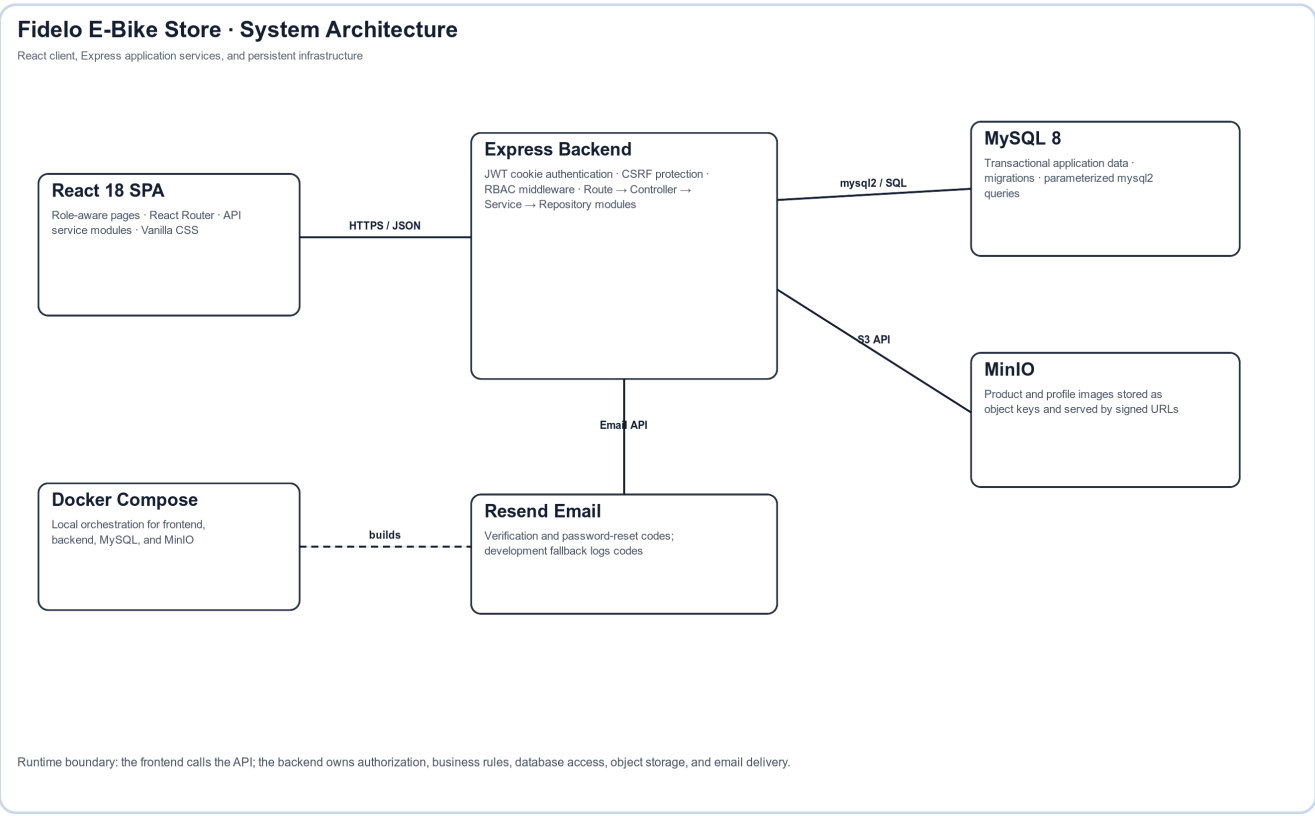
- Maintain accurate product quantities for branches and warehouses.
- Provide traceable stock adjustments and transfers.
- Let customers browse products and submit applications for offline transactions.
- Apply role and location boundaries consistently.
- Give staff timely low-stock and workflow notifications.

2. Scope and Users

Role	Primary scope
Administrator	System-wide users, locations, catalog, inventory, transfers, applications, reports, and audit logs.
Branch staff	Assigned-branch inventory, applications, transfers, notifications, and reports.
Warehouse staff	Assigned-warehouse inventory, movements, transfers, notifications, and reports.
Customer	Public catalog, product applications, application tracking, notifications, and profile.

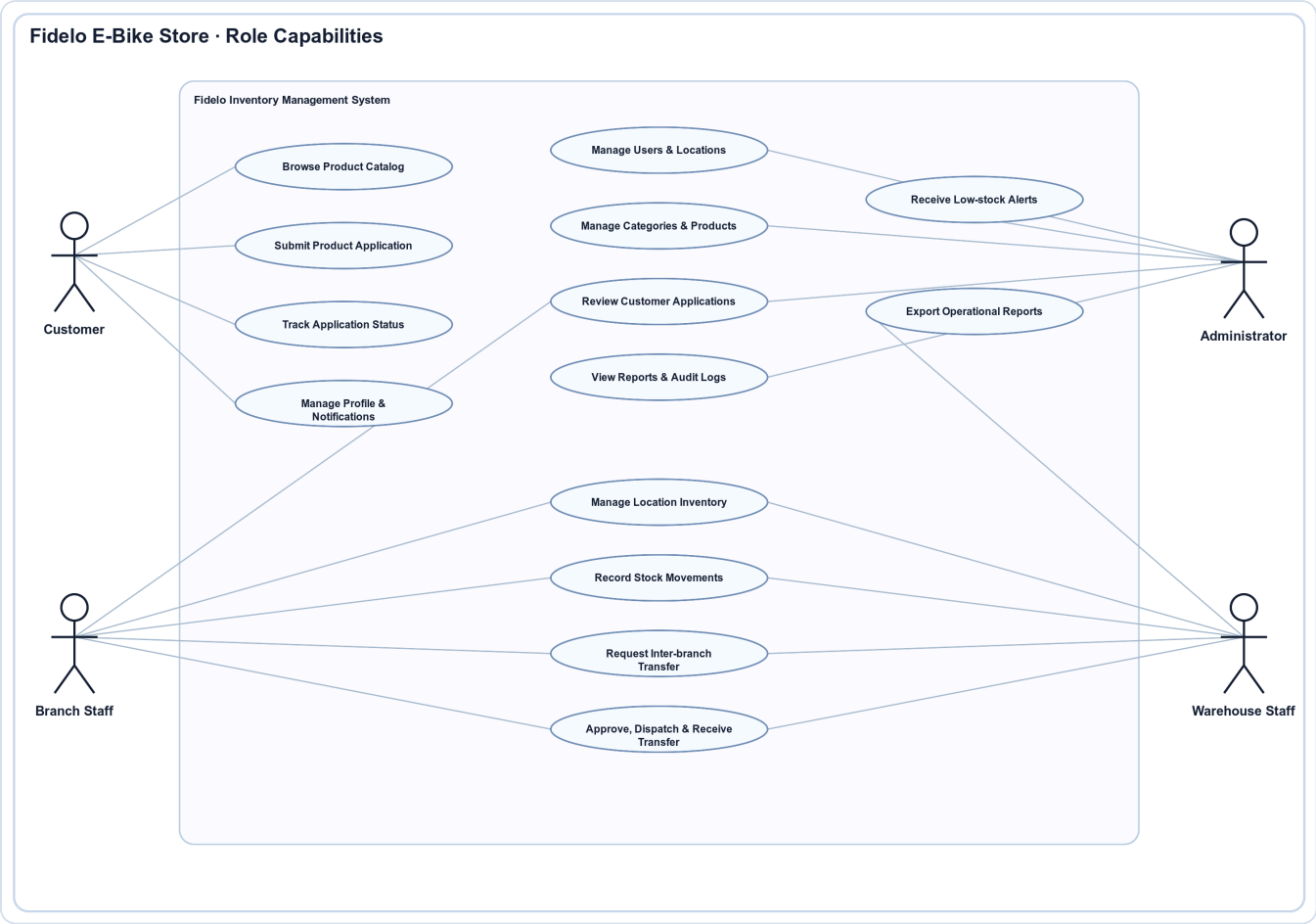
Sales payment processing, delivery logistics, and online payment gateways are outside the implemented scope.

3. System Architecture



The React 18 single-page application communicates with the Express API through reusable frontend service modules. Backend features follow Route → Controller → Service → Repository → MySQL. MinIO stores product and avatar objects, while Resend supports verification and password-reset email. Docker Compose provides the four local runtime services.

4. Functional Capabilities



Catalog and inventory

Administrators manage categories, products, and locations. Authorized staff inspect inventory, configure low-stock thresholds, and record stock-in, stock-out, and adjustment movements. Every movement stores before and after quantities and the responsible user.

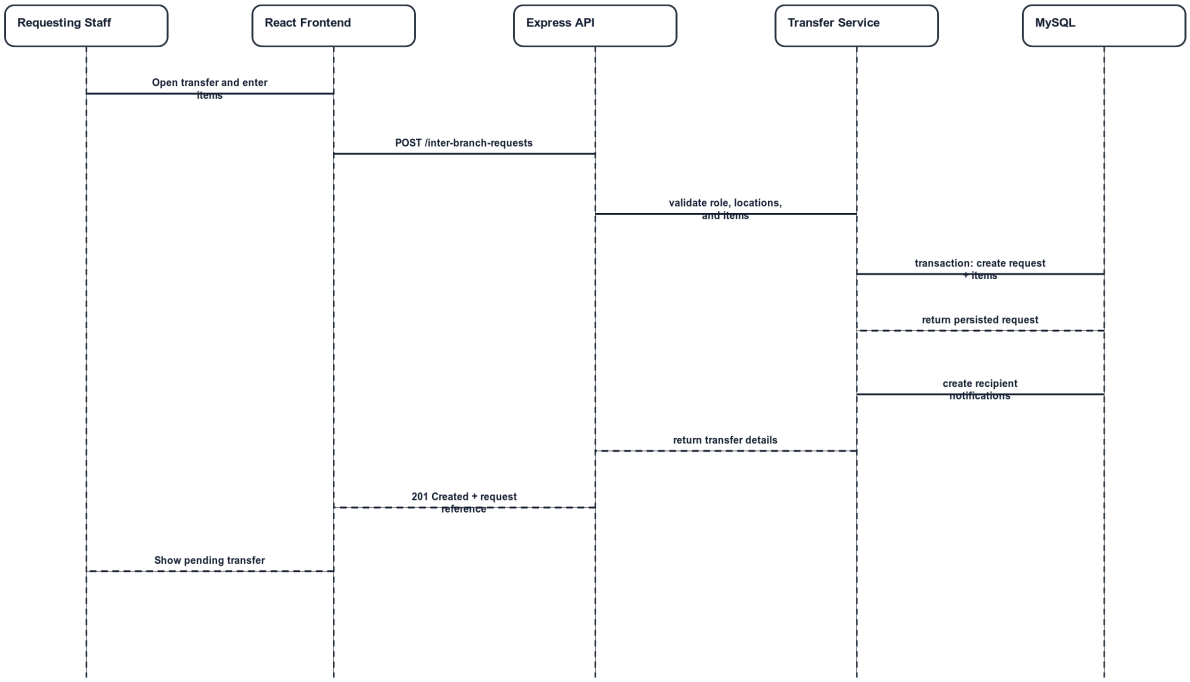
Customer applications

A verified customer selects a product and location, submits required contact and identity details, and tracks the application through submitted, under review, approved or rejected, and completed or cancelled states. Authorized branch staff or administrators review the record. Sensitive identity values are handled by the backend encryption utilities.

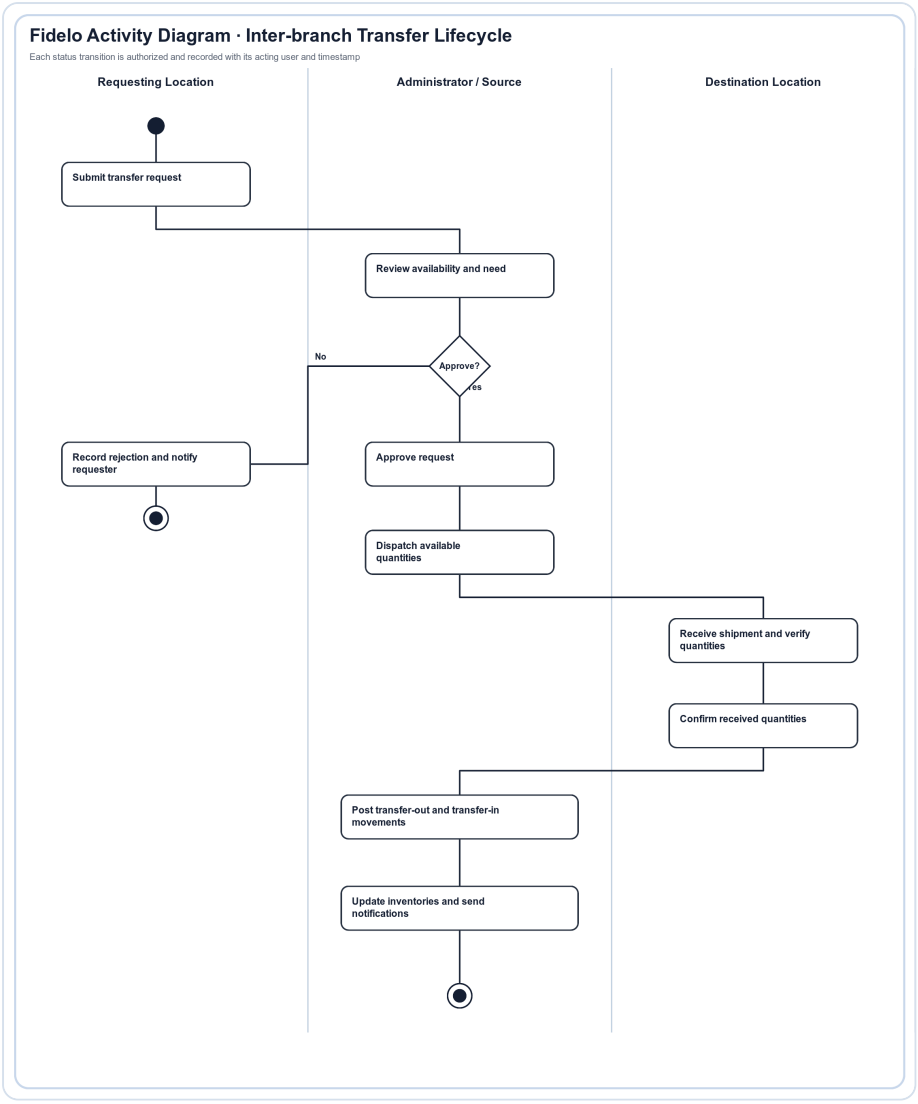
5. Transfer Workflow

Fidelo Sequence Diagram · Request Stock Transfer

Creation path through the required Route → Controller → Service → Repository tiers

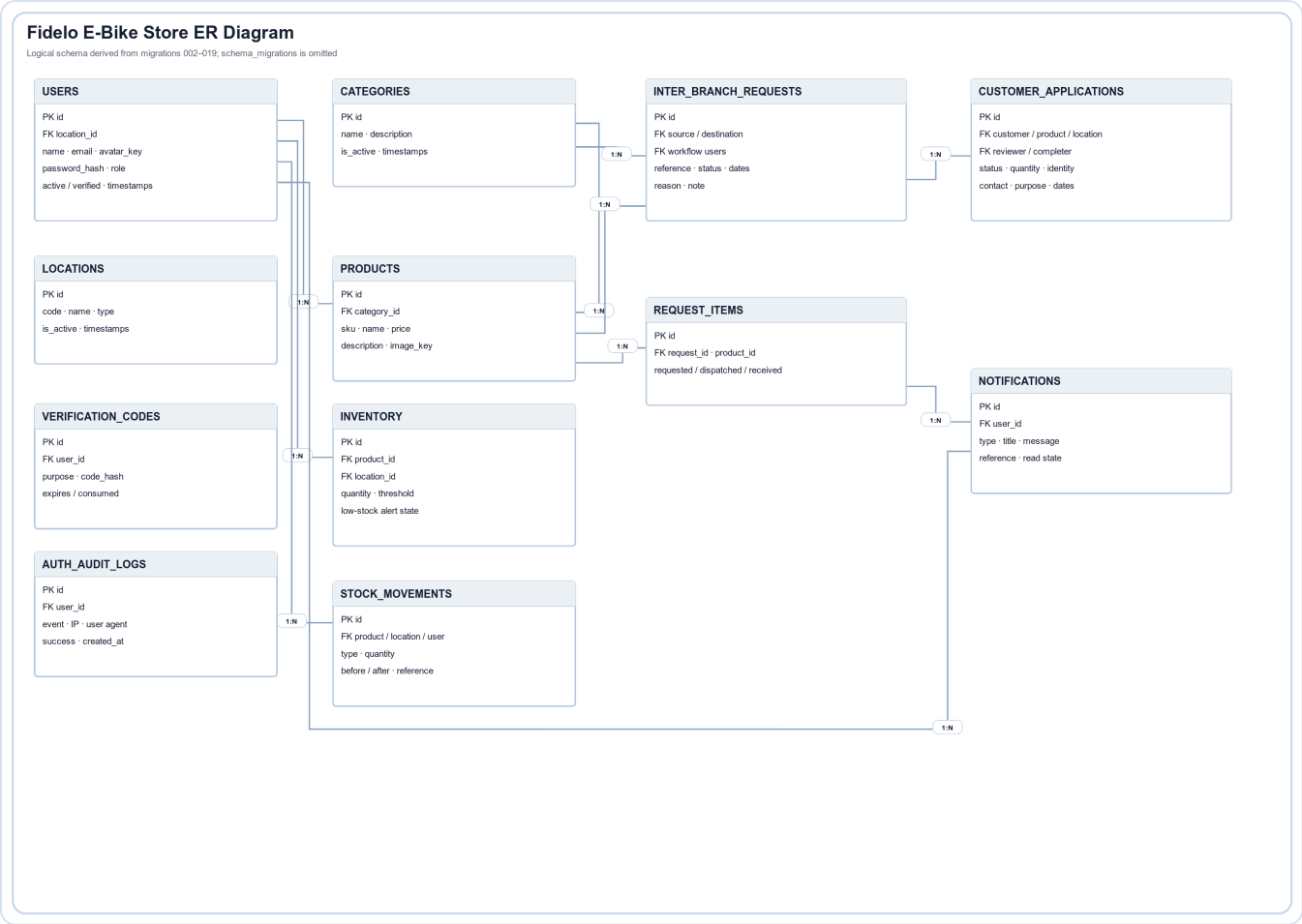


Result: a pending request is committed atomically and relevant users receive an in-app notification.



A transfer begins as pending. An authorized reviewer may approve or reject it. Approved stock is dispatched from the source and later received at the destination. The service validates status transitions and quantities, writes the corresponding transfer-out and transfer-in ledger entries, updates both inventory balances, and creates notifications.

6. Data Model



The database uses foreign keys and check constraints to maintain referential and quantity integrity. Inventory is unique per product and location. Movement rows are append-only history. Transfer requests and customer applications retain workflow actors and timestamps for accountability.

7. API and Backend Modules

Base path	Responsibility
/auth	Registration, login/logout, verification, password reset, and current session.
/users	Profile and administrator user management.
/locations, /categories, /products	Master data and public catalog.
/inventory	Balances, thresholds, and stock movements.
/inter-branch-requests	Transfer creation and lifecycle actions.
/applications	Customer applications and staff review.
/notifications	Unread counts, lists, and read state.
/reports	Role-scoped operational summaries.
/uploads	Authorized object uploads.

All endpoints are mounted below `/api/v1`. Database access remains in repositories and uses parameterized placeholders.

8. Security and Controls

- JWT authentication is carried in an HTTP-only cookie.
- CSRF tokens protect state-changing browser requests.
- Role-based middleware and service checks enforce permissions and location scope.
- Passwords and verification codes are stored as hashes.
- Selected customer identity fields use application-level encryption.
- Authentication events retain success, IP address, user agent, and timestamp.
- Uploads use validated object-storage operations instead of public filesystem paths.
- Configuration and secrets are read through environment-backed config.

9. Deployment and Operations

The supported local workflow is:

```
docker compose up -d --build
```

Service	Default endpoint
Frontend	http://localhost:3000
Backend API	http://localhost:5000/api/v1
MySQL	localhost:3306
MinIO API / console	localhost:9000 / localhost:9001

The backend entrypoint applies ordered SQL migrations and development seed routines before starting the API. MySQL and MinIO data live in named Docker volumes.

10. Verification and Maintenance

Backend behavior is covered by Node test suites for inventory, transfer requests, customer applications, and notifications. The frontend is verified through its Vite production build. Documentation should be regenerated after material changes to roles, services, workflows, or migrations.

```
docker compose exec backend npm test
docker compose up -d --build
```

Document basis: repository implementation and SQL migrations through migration 019, reviewed 15 August 2026.